

Agent Execution Engine - Implementation Documentation

Date: November 17, 2025

Version: 1.0

Status:  Completed

Overview

This document describes the implementation of the backend agent execution engine for Mindous.ai, following the architecture specified in `/home/ubuntu/mindous/AGENT_ARCHITECTURE.md`.

Implementation Summary

Completed Components

1. Database Schema Updates

- Created migration file: `db/migrations/0005_romantic_charles_xavier.sql`
- New tables: `builds`, `build_artifacts`, `code_generations`, `execution_state`
- Drizzle schema files created with proper relations and indexes

2. Event Publisher Service (`/lib/services/event-publisher.ts`)

- Real-time event streaming via Redis
- Support for agent-specific events (code generation, builds, previews)
- Database persistence for event replay
- Type-safe event publishing methods

3. Code Generator Service (`/lib/services/code-generator.ts`)

- LLM-powered code generation
- Support for components, pages, APIs, utilities
- Framework-aware generation (React, Next.js)
- Language support: TypeScript, JavaScript, CSS, JSON, HTML
- Context-aware prompting with dependencies and project structure

4. Agent Execution Engine (`/lib/agents/execution-engine.ts`)

- Core autonomous execution loop
- Task decomposition using existing planning service
- Step-by-step execution with state management
- Code generation integration
- Event streaming for real-time progress
- Error handling and recovery
- Cancellation support

5. API Routes

- `POST /api/agent/execute` - Start agent execution
- `GET /api/agent/status/[executionId]` - Get execution status
- `POST /api/agent/stop/[executionId]` - Stop/cancel execution
- Enhanced SSE streaming (already compatible with new events)

6. Tool Framework Extensions

- New tool manifests:
 - `code_generation` - AI-powered code generation
 - `file_operations` - File system operations
 - `build_tool` - Project building (Next.js, React)
 - Updated tool registry exports

7. Test Infrastructure

- Test script: `/scripts/test-agent-engine.ts`
- Verifies end-to-end agent execution flow

Database Schema

New Tables

`builds`

Tracks build operations for generated projects.

Column	Type	Description
id	UUID	Primary key
run_id	UUID	Foreign key to runs
execution_id	UUID	Foreign key to executions
user_id	TEXT	User identifier
project_name	TEXT	Project name
project_type	ENUM	nextjs, react, html, nodejs, other
status	ENUM	queued, installing, building, completed, failed, cancelled
build_path	TEXT	File system path
output_path	TEXT	Build output path
build_logs	TEXT	Build logs
error_message	TEXT	Error message if failed
started_at	TIMESTAMP	Build start time
completed_at	TIMESTAMP	Build completion time
duration_ms	INTEGER	Build duration
size_bytes	BIGINT	Build size
metadata	JSONB	Additional metadata

build_artifacts

Stores generated code files and assets.

Column	Type	Description
id	UUID	Primary key
build_id	UUID	Foreign key to builds
file_path	TEXT	Relative file path
file_type	TEXT	File type (tsx, ts, css, etc.)
content	TEXT	File content
storage_path	TEXT	Supabase storage path
size_bytes	INTEGER	File size
mime_type	TEXT	MIME type
is_generated	INTEGER	Whether file was AI-generated
metadata	JSONB	Additional metadata

code_generations

Tracks individual code generation requests.

Column	Type	Description
id	UUID	Primary key
run_id	UUID	Foreign key to runs
subtask_id	UUID	Foreign key to subtasks
user_id	TEXT	User identifier
prompt	TEXT	Generation prompt
generated_code	TEXT	Generated code
language	TEXT	Programming language
framework	TEXT	Framework (react, nextjs, etc.)
llm_provider	TEXT	LLM provider used
llm_model	TEXT	LLM model used
tokens_used	INTEGER	Tokens consumed
generation_time_ms	INTEGER	Generation duration
validation_status	ENUM	valid, invalid, unchecked
validation_errors	JSONB	Validation error details
metadata	JSONB	Additional metadata

execution_state

Stores agent execution state for recovery and debugging.

Column	Type	Description
id	UUID	Primary key
execution_id	UUID	Foreign key to executions
run_id	UUID	Foreign key to runs
current_step	TEXT	Current step name
step_index	INTEGER	Current step index
total_steps	INTEGER	Total number of steps
context	JSONB	Execution context variables
variables	JSONB	Runtime variables
artifacts	JSONB	Generated artifacts references
decisions	JSONB	Agent decision history

Architecture Components

1. Event Publisher Service

Location: `/lib/services/event-publisher.ts`

Responsibilities:

- Publish events to Redis for real-time streaming
- Store events in database for replay
- Provide type-safe event publishing methods
- Support agent-specific event types

Event Types:

```
// Execution lifecycle
EXECUTION_STARTED
EXECUTION_PROGRESS
EXECUTION_COMPLETED
EXECUTION_FAILED

// Step events
STEP_STARTED
STEP_PROGRESS
STEP_COMPLETED

// Tool events
TOOL_CALLED
TOOL_RESULT

// Code generation events
CODE_GENERATION_STARTED
CODE_GENERATION_CHUNK
CODE_GENERATION_COMPLETED

// Build events
BUILD_STARTED
BUILD_LOG
BUILD_PROGRESS
BUILD_COMPLETED
BUILD_FAILED

// Preview events
PREVIEW_STARTING
PREVIEW_READY
PREVIEW_FAILED
```

Key Methods:

```
EventPublisher.publishExecutionStarted(runId, data)
EventPublisher.publishExecutionProgress(runId, data)
EventPublisher.publishCodeChunk(runId, subtaskId, chunk)
EventPublisher.publishBuildLog(runId, log)
EventPublisher.publishPreviewReady(runId, data)
```

2. Code Generator Service

Location: `/lib/services/code-generator.ts`

Responsibilities:

- Generate code using LLM
- Support multiple languages and frameworks
- Context-aware code generation
- File name suggestion
- Code extraction from LLM responses

Key Functions:

```
generateCode(request: CodeGenerationRequest): Promise<CodeGenerationResult>
generateNextJSComponent(params): Promise<CodeGenerationResult>
generateAPIRoute(params): Promise<CodeGenerationResult>
generateMultipleFiles(requests[]): Promise<CodeGenerationResult[]>
```

Supported Types:

- `component` - React/Next.js components
- `page` - Next.js pages
- `api` - API routes
- `utility` - Utility functions
- `config` - Configuration files
- `style` - CSS/styling files

3. Agent Execution Engine

Location: `/lib/agents/execution-engine.ts`

Responsibilities:

- Orchestrate entire execution flow
- Task decomposition and planning
- Step-by-step execution
- Code generation coordination
- State management
- Event streaming
- Error handling and recovery

Execution Flow:

1. Create execution **and** **run** records
2. Publish execution started event
3. Plan execution (decompose **into** **steps**)
4. **For** each **step**:
 - a. Create subtask
 - b. Publish **step** started event
 - c. Execute **step** action
 - d. Store results
 - e. Publish **step** completed event
 - f. Update progress
5. Mark execution as completed
6. Publish completion event

Key Methods:

```
class AgentExecutionEngine {
  async executeTask(params): Promise<ExecutionResult>
  private async planExecution(prompt): Promise<ExecutionPlan>
  private async executeSteps(plan): Promise<void>
  private async executeStep(title, description, type): Promise<any>
  async cancel(): Promise<void>
  getStatus(): ExecutionStatus
}
```

Step Types Handled:

- Analysis/Research - Uses LLM to analyze and gather information
- Code Generation - Generates components, pages, APIs
- Tool Execution - Calls external tools
- Review - Validates and reviews outputs

4. API Routes

POST /api/agent/execute

Start a new agent execution.

Request:

```
{
  "prompt": "Create a todo app with Next.js",
  "context": {
    "taskType": "code",
    "complexity": "medium",
    "constraints": ["Use TypeScript", "Use Tailwind CSS"]
  }
}
```

Response:

```
{
  "success": true,
  "executionId": "uuid",
  "runId": "uuid",
  "message": "Agent execution started"
}
```

GET /api/agent/status/[executionId]

Get execution status and details.

Response:

```
{
  "success": true,
  "execution": {
    "id": "uuid",
    "status": "running",
    "startTime": "2024-01-01T00:00:00Z",
    "endTime": null
  },
  "run": {
    "id": "uuid",
    "status": "running",
    "totalSteps": 5,
    "completedSteps": 2,
    "progress": 40
  },
  "currentState": {
    "currentStep": "Generating component",
    "stepIndex": 2,
    "totalSteps": 5,
    "progress": 40
  },
  "subtasks": [...],
  "artifacts": [...]
}
```

POST /api/agent/stop/[executionId]

Stop/cancel an ongoing execution.

Response:

```
{
  "success": true,
  "message": "Execution stopped",
  "executionId": "uuid"
}
```

5. Tool Framework

New Tools:

1. Code Generation Tool

- Key: `code_generation`
- Generates code using AI
- Supports multiple languages and frameworks

2. File Operations Tool

- Key: `file_operations`
- Performs file system operations (CRUD)
- Supports directory creation

3. Build Tool

- Key: `build_tool`
- Builds Next.js/React projects
- Installs dependencies
- Captures build logs

Usage Example

Starting an Agent Execution

```
// API Call
const response = await fetch('/api/agent/execute', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    prompt: 'Create a user profile component with avatar and bio',
    context: {
      taskType: 'code',
      complexity: 'low',
      constraints: ['Use TypeScript', 'Use Tailwind CSS']
    }
  })
});

const { executionId, runId } = await response.json();
```

Streaming Progress

```
// Subscribe to SSE stream
const eventSource = new EventSource(`/api/streams/runs/${runId}`);

eventSource.addEventListener('event', (event) => {
  const data = JSON.parse(event.data);

  switch (data.eventType) {
    case 'EXECUTION_STARTED':
      console.log('Execution started');
      break;

    case 'CODE_GENERATION_CHUNK':
      console.log('Code chunk:', data.data.chunk);
      break;

    case 'STEP_COMPLETED':
      console.log('Step completed:', data.message);
      break;

    case 'EXECUTION_COMPLETED':
      console.log('Execution completed!');
      eventSource.close();
      break;
  }
});
```

Checking Status

```
const statusResponse = await fetch(`/api/agent/status/${executionId}`);
const status = await statusResponse.json();

console.log(`Progress: ${status.run.progress}%`);
console.log(`Current Step: ${status.currentState.currentStep}`);
console.log(`Artifacts: ${status.artifacts.length}`);
```

Testing

Running the Test

```
cd /home/ubuntu/mindous
npx tsx scripts/test-agent-engine.ts
```

The test will:

1. Create test task, execution, and run records
2. Start agent execution with a simple prompt
3. Monitor progress
4. Verify database state
5. Display generated artifacts

Expected Output

```

 Testing Agent Execution Engine...

1  Creating test task...
 Task created: <uuid>

2  Creating execution record...
 Execution created: <uuid>

3  Creating run record...
 Run created: <uuid>

4  Starting agent execution...
=====

 Starting agent execution for run <uuid>
 Planning execution...
 Created execution plan with 3 steps
 Executing step 1/3: Create Counter Component
 Generating component: Create Counter Component
 Completed step 1/3: Create Counter Component
 Executing step 2/3: Add Styling
...

=====

5  Agent execution completed 

 Execution Results:
Status: completed
Execution ID: <uuid>
Run ID: <uuid>

 Generated Artifacts (1):
1. counter-component.tsx (code)
   Preview (first 200 chars):
   import React, { useState } from 'react';

   export default function Counter() {
     const [count, setCount] = useState(0);
     ...

```

Issues and Considerations

Known Issues

1. Migration Not Applied

- The database migration needs to be applied before running the agent
- Run: `npx drizzle-kit push` or apply migration manually

2. Background Execution

- Agent executions run in the background
- Consider implementing a job queue (BullMQ, etc.) for production
- Current implementation may time out on serverless platforms

3. Code Validation

- Generated code is not validated (syntax checking)
- Consider adding ESLint/TypeScript validation

4. Build System Not Implemented

- Build service and preview deployment are not yet implemented
- These are defined in the architecture but need separate implementation

Future Enhancements

1. Streaming Code Generation

- Implement token-by-token streaming for code generation
- Use LLM streaming capabilities

2. Build System

- Implement build service for Next.js/React projects
- Add preview deployment with isolated environments
- Port management for preview URLs

3. Error Recovery

- Add retry logic for failed steps
- Implement checkpoint/resume functionality
- Better error messages and suggestions

4. Performance Optimization

- Cache common code patterns
- Parallel execution of independent steps
- Resource pooling

5. Security

- Sandbox code execution
- Rate limiting
- Input validation and sanitization

6. Monitoring

- Add metrics collection
 - Performance tracking
 - Cost analysis
-

Dependencies

Required Environment Variables

```
# LLM Providers
OPENAI_API_KEY=sk-proj-...
ANTHROPIC_API_KEY=sk-ant-...
ABACUSAI_API_KEY=s2_...

# Database
DATABASE_URL=postgresql://...

# Redis (for event streaming)
REDIS_URL=redis://...

# Clerk (for authentication)
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=pk_...
CLERK_SECRET_KEY=sk_...

# Supabase (for file storage - optional)
NEXT_PUBLIC_SUPABASE_URL=https://...
SUPABASE_SERVICE_ROLE_KEY=eyJ...
```

Package Dependencies

All required packages are already installed:

- drizzle-orm - Database ORM
- ioredis - Redis client
- @clerk/nextjs - Authentication
- zod - Schema validation
- openai , @anthropic-ai/sdk , @google/generative-ai - LLM clients

Deployment Checklist

Before deploying to production:

- [] Apply database migration: `npx drizzle-kit push`
 - [] Set all required environment variables
 - [] Test agent execution with sample prompts
 - [] Verify SSE streaming works
 - [] Set up Redis for production (Redis Cloud, Upstash, etc.)
 - [] Configure rate limiting
 - [] Set up error monitoring (Sentry, etc.)
 - [] Test authentication and authorization
 - [] Configure CORS if needed
 - [] Set up logging and monitoring
-

File Structure

/home/ubuntu/mindous/		
db/		
schema/		
builds.ts	✓	NEW
code-generations.ts	✓	NEW
execution- state .ts	✓	NEW
index .ts	✓	UPDATED
migrations/		
0005_romantic_charles_xavier.sql	✓	NEW
lib/		
agents/		
execution-engine.ts	✓	NEW
services/		
event-publisher.ts	✓	NEW
code-generator.ts	✓	NEW
planning-service.ts	✓	EXISTING
tool-execution-service.ts	✓	EXISTING
tools/		
code-generation.ts	✓	NEW
file -operations.ts	✓	NEW
build.ts	✓	NEW
index .ts	✓	UPDATED
app/api/		
agent/		
execute /route.ts	✓	NEW
status/[executionId]/route.ts	✓	NEW
stop/[executionId]/route.ts	✓	NEW
streams/		
runs/[runId]/route.ts	✓	EXISTING (Compatible)
scripts/		
test-agent-engine.ts	✓	NEW

Conclusion

The backend agent execution engine has been successfully implemented with all core components in place:

- ✓ Database schema and migrations
- ✓ Event publishing service
- ✓ Code generation service
- ✓ Agent execution engine
- ✓ API routes
- ✓ Tool framework extensions
- ✓ Test infrastructure

The system is ready for integration testing. The next steps would be:

1. Apply database migration
2. Run integration tests
3. Implement build system and preview deployment (Phase 2)
4. Add frontend components for agent interaction
5. Deploy to staging environment

Last Updated: November 17, 2025

Implemented By: AI Assistant

Review Status: Pending review by development team